

SOLID Principles in Java (Online Shopping Example)

1. Single Responsibility Principle (SRP)

A class should have only one reason to change.

Violation:

```
class UserService {
    public void registerUser(String name, String email) {
        // save to DB
        sendWelcomeEmail(email);
    }
    private void sendWelcomeEmail(String email) {
        ...
    }
}
```

Fix:

```
class UserService {
    public void registerUser(String name, String email) {
        // save to DB
    }
}
class EmailService {
    public void sendWelcomeEmail(String email) {
        ...
    }
}
```

2. Open/Closed Principle (OCP)

Software entities should be open for extension, but closed for modification.

Violation:

```
class PaymentService {
    public void pay(String method) {
        if (method.equals("CreditCard")) ...
        else if (method.equals("PayPal")) ...
    }
}
```

Fix:

```
interface PaymentMethod {
    void pay();
}
class CreditCardPayment implements PaymentMethod {
    public void pay() { ... }
}
class PayPalPayment implements PaymentMethod {
    public void pay() { ... }
}
class PaymentService {
```

SOLID Principles in Java (Online Shopping Example)

```
        public void makePayment(PaymentMethod method) {  
            method.pay();  
        }  
    }  
}
```

3. Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types.

Violation:

```
class Product {  
    public double getPrice() { return 100.0; }  
}  
class FreeProduct extends Product {  
    public double getPrice() { throw new UnsupportedOperationException(); }  
}
```

Fix:

```
abstract class Product {  
    public abstract double getPrice();  
}  
class StandardProduct extends Product {  
    public double getPrice() { return 100.0; }  
}  
class DiscountedProduct extends Product {  
    public double getPrice() { return 80.0; }  
}
```

4. Interface Segregation Principle (ISP)

Clients should not depend on interfaces they do not use.

Violation:

```
interface UserOperations {  
    void placeOrder();  
    void manageInventory();  
}  
class Customer implements UserOperations {  
    public void placeOrder() {}  
    public void manageInventory() { throw new UnsupportedOperationException(); }  
}
```

Fix:

```
interface OrderPlacer { void placeOrder(); }  
interface InventoryManager { void manageInventory(); }  
class Customer implements OrderPlacer {  
    public void placeOrder() { ... }  
}  
class Admin implements InventoryManager {  
    public void manageInventory() { ... }  
}
```

SOLID Principles in Java (Online Shopping Example)

```
}
```

5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules.

Violation:

```
class FixedDiscount {
    public double applyDiscount(double amount) { return amount - 10; }
}
class ShoppingCart {
    FixedDiscount discount = new FixedDiscount();
    public double checkout(double total) {
        return discount.applyDiscount(total);
    }
}
```

Fix:

```
interface DiscountService {
    double applyDiscount(double amount);
}
class FixedDiscountService implements DiscountService {
    public double applyDiscount(double amount) { return amount - 10; }
}
class ShoppingCart {
    private final DiscountService discountService;
    public ShoppingCart(DiscountService discountService) {
        this.discountService = discountService;
    }
    public double checkout(double total) {
        return discountService.applyDiscount(total);
    }
}
```